

Bad Smells in Code

Software Re-Engineering CS4118

Lect 7

Software design defects

Terminology

- Design defects / technical debt / code smells / Bug Density
 - Violations of design principles but not necessarily a functional defect (bug)
 - Anti-patterns
- Common solutions to recurring problems that do more harm than good
 - Coined by Andrew Koenig, inspired by the idea of Design Patterns

Duplicated Code

- same expression in two methods of the same class
- same expression in two sibling subclasses
- code is similar but not the same
- methods do the same thing with a different algorithm
- duplicated code in two unrelated classes

Long Method

- longer a procedure is, the more difficult it is to understand.
- method with lots of parameters and temporary variables
- Conditionals and loops also make long methods
- Ideal LOC in a method?

Large Class

- a class is trying to do too much
 - Too many instance variables, duplicated code
- a class with too much code
- a class with a huge wad of variables

Long Parameter List

- long parameter lists are hard to understand
- important exception
 - when you explicitly do not want to create a dependency from the called object to the larger object
 - Unpacking data and sending it along as parameters is reasonable

Divergent Change

- one class is commonly changed in different ways for different reasons.
- identify everything that changes for a particular cause
- Use Extract Class to put them all together.

Shotgun Surgery

- every time you make a kind of change, you have to make a lot of little changes to a lot of different classes
 - Opposite of divergent change
 - easy to miss an important change.
- ideally, there is a one-to-one link between common changes and classes

Feature Envy

- a method that seems more interested in a class other than the one it actually is in
 - The most common focus of the envy is the data
- a method uses features of several classes
 - determine which class has most of the data and put the method with that data
- Strategy and Visitor break this rule
 - allow you to change behavior easily at the cost of further indirection.

Data Clumps

- Bunches of data that hang around together really ought to be made into their own object
- A good test is to consider deleting one of the data values
 - would the others make any sense?
- once you have the objects, look for cases of feature envy

Primitive Obsession

- two kinds of data
 - Record types to structure data into meaningful groups
 - Primitive types are the building blocks
- Records always carry a certain amount of overhead
- Objects blur or even break the line between primitive and larger classes
- a group of fields that should go together
- primitives in parameter lists
- Handling an array

Switch Statements

- Good OO code lacks switch statements
- switch statements causes duplication
- polymorphism should replace switch statement
- Replace Type Code with Subclasses or Replace Type Code with State/Strategy
- When you have set up the inheritance structure
 - Replace Conditional with Polymorphism
- When polymorphism is overkill
- If a conditional cases is a null
 - Null Object

Parallel Inheritance Hierarchies

- Every time you make a subclass of one class, you also have to make a subclass of another
 - a special case of shotgun surgery

Lazy Class

- A class that isn't doing enough should be eliminated
 - Often a class that has been downsized with refactoring
 - Or a class that was added because of changes that were planned but not made
- If subclasses aren't doing enough
 - Collapse Hierarchy
- Nearly useless components
 - Inline Class

Speculative Generality

- When people say, "Oh, I think we need the ability to this kind of thing someday"
 - want all sorts of hooks and special cases to handle things that aren't required
 - often is harder to understand and maintain
- abstract classes that aren't doing much
- Unnecessary delegation
- Methods with unused parameters
- Methods named with odd abstract names
 - Rename Method

Temporary Field

- an object in which an instance variable is set only in certain circumstances
- when a complicated algorithm needs several variables
 - Some fields are valid only during the algorithm
 - Extract Class with these variables and the methods that require them.
 - The new object is a method object

Message Chains

- when a client asks one object for another object
 - which the client then asks for yet another object
 - which the client then asks for yet another object
 - and so on
- A better alternative is to see what the resulting object is used for
- If several clients of one of the objects in the chain want to navigate the rest of the way
 - add a method to do that

Inappropriate Intimacy

- Sometimes classes become far too intimate
- Change Bidirectional Association to Unidirectional
- If the classes have common interests
 - Extract Class to put the commonality
- Inheritance often can lead to over intimacy
 - Subclasses know more about their parents

Incomplete Library Class

- Library builders have a really tough job
 - bad form
 - usually impossible to modify a library class
- Special-purpose tools for this
- If there are just a couple of methods that you wish the library class had
 - Introduce foreign method

Data Class

- Classes that have fields, getting and setting methods for the fields, and nothing else
 - Manipulated by other classes
- May have public fields
 - Encapsulate Field
- If you have collection fields
 - see whether they are properly encapsulated
 - Encapsulate Collection if they aren't
 - Remove Setting Method on any field that should not be changed
- to move behavior into the data class
 - Move Method
- If you can't move a whole method
 - Extract Method to create a method that can be moved
 - Hide Method on the getters and setters
- Data classes are like children
 - They are okay as a starting point, but to participate as a grownup object, they need to take some responsibility

Refused Bequest

- Subclasses get to inherit the methods and data of their parents
 - what if they don't want or need what they are given?
- this means the hierarchy is wrong
- Special Case: the subclass is reusing behavior but does not want to support the interface of the superclass

Comments

- If you need a comment to explain what a block of code does
 - Extract Method
- If the method is already extracted but you still need a comment to explain what it does
 - Rename Method

Tip

When you feel the need to write a comment, first try to refactor the code so that any comment becomes superfluous.

Other code smells

Code Smell	Description
Duplicated Code	Repetition of same code lines
Conditional Complexity	Too many if-else / conditional logic blocks
Combinatorial explosion	Almost similar code with minor variations in data / behavior
Primitive obsession	Too much reliance on primitive data types
Data class	Classes with only attributes and accessors (similar to anemic domain model)
Data clumps	Data members that are always found together
Refused bequest	Limited reliance on inherited features
Feature envy	Methods that extensively use another class(es)
Message chains	Long sequences of method calls
Middle man	Classes that simply act as intermediaries in communication
Shotgun surgery	Change in one class requires cascading changes in many
Parallel inheritance hierarchies	Introducing subclass of one necessitates subclass of another

Metrics and heuristics

Code Smell - Identification mechanisms

Conditional complexity	High cyclomatic complexity
Feature envy, Middle man, Shotgun surgery	High RFC, CBO
Duplicated code, Combinatorial explosion	Relies on code reviews / static analysis tools
Primitive obsession	Heuristic-based: Parallel arrays / attributes are usually an indicator
Data clumps	Heuristic-based, relies on code review
Refused bequest	Heuristic-based: Inappropriate inheritance that seems semantically incorrect
Message chains	Heuristic-based, relies on code reviews / static analysis
Parallel inheritance heirarchies	Classes with same NOC (number of children)